

Examen 17 de Junio 2016

TEORIA:

Generación del código intermedio

Consiste en el primer paso dentro de la fase de síntesis, dentro de la estructura de un compilador. En este caso genera una representación intermedia de código con las siguientes características:

- Fácil de generar a partir de código fuente
- Fácil de traducir al código ejecutable
- Es una fase opcional pero muy recomendable
- Se utilizan definiciones dirigidas por la sintaxis o esquemas de traducción. Tipos:
 - Notación postfija
 - Árboles sintácticos
 - Grafos dirigidos acíclicos
 - Código de 3 direcciones: Cuádruplas, Triples y Triples indirectos.

Bootstrapping

- Descripción: Consiste en un proceso con el cual podremos a través de la combinación de compiladores más “simples” y “sencillos” obtener compiladores más complicados de implementar. A la hora de llevar acabo esto debemos de tener en cuenta que los compiladores se componen de 3 lenguajes:
 - Lenguaje Fuente (F): Lenguaje del fichero fuente a utilizar.
 - Lenguaje de Implementación (I): Lenguaje con el cual esta escrito el compilador.
 - Lenguaje Objeto (O): Lenguaje del fichero obtenido tras aplicar la compilación.
- A la hora de utilizar el bootstrapping, lo que se busca es a través de combinaciones de compiladores obtener más compiladores. Una expresión general para entender este funcionamiento sería: $F_I O + I_M N$ generando así el $F_N O$
- Ejemplos
 - Algunos ejemplos concretos serían en el caso de que $F = I$ significaría que nos encontramos con un autocompilador, en este caso este compilador deberá de ser compilado para poder hacer uso.
 - $I \neq 0$ Si ocurre esto, estamos hablando de compilador cruzado, en este caso este compiladores generará código para otra máquina.

Análisis léxico: Tipos de reconocimiento de las palabras claves

- Implícito: En este caso las palabras claves se encuentran preescritas en la tabla de símbolos, en este caso su reconocimiento es más lento, pero simplifica mucho más la implementación del analizador léxico.
- Explícito: En este caso se definen reglas para reconocer cada una de las palabras reservadas del lenguaje, minimizando así el tiempo para reconocer cada una de las palabras. El problema reside en la complejidad y que dificulta más la implementación del analizador.

Criterios “generales” para el tratamiento de errores léxicos

- **Informar del error** ->Localización del error y descripción del error, indicando motivo y características
- **Evitar cascada de errores** ->Informar del error solo una vez, aunque se produzca más veces
- **Reparar el error** ->Reparar un error encontrado y si es posible avisar de la corrección realizada
- **Continuar con el proceso de traducción** ->Para encontrar más errores

EJERCICIOS:**Componentes léxicos**

- La siguiente expresión regular denota números naturales: $c + n (c + n)^*$ donde:
 - c es el número 0
 - n es un dígito del 1 al 9
- Utiliza el **Algoritmo de Thompson** para construir un AFN equivalente a la expresión regular.
- Utiliza el **Algoritmo de Construcción de Subconjuntos** para obtener un AFD equivalente al AFN obtenido en el apartado anterior.
- Minimiza**, si es posible, el AFD obtenido en el apartado anterior.
- Comprueba si el último autómata obtenido reconoce la sentencia: $n n c c$

En primer lugar deberemos de aplicar el algoritmo de Thompson, con el cual siguiendo los pasos de la **Figura 1 y 2**. Una vez realizado esto, deberemos de aplicar el algoritmo de Construcción de Subconjuntos.

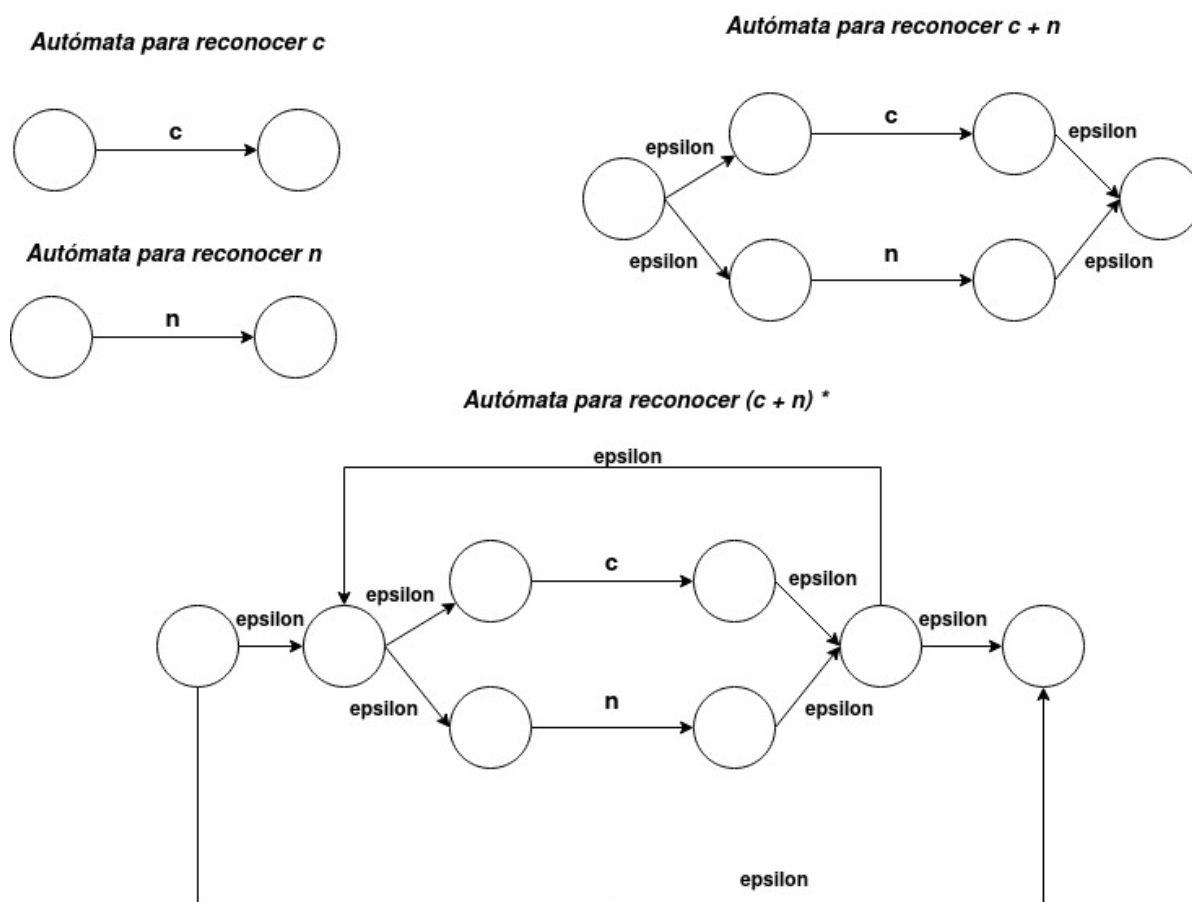


Figura 1: Autómatas Iniciales para generar el autómata final

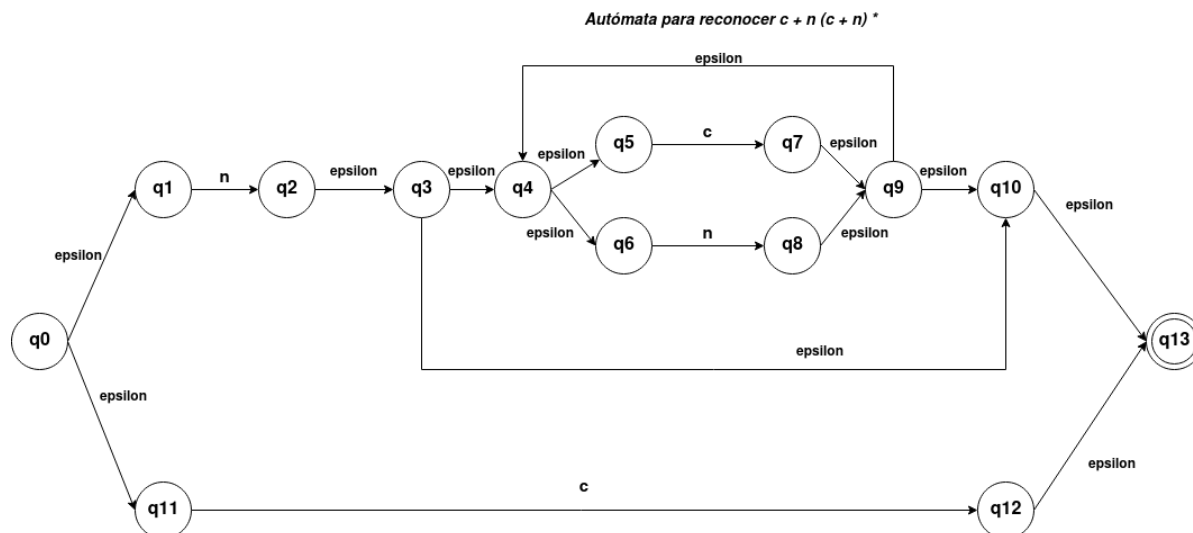


Figura 2: Autómata Final de la expresión $c + n (c + n)^*$

Construcción de subconjuntos:

En primer lugar deberemos de realizar la clausura- ϵ del estado inicial (q_0).

$$p_0 = \text{clausura} - \epsilon(q_0) = \{q_0, q_1, q_{11}\} \quad (1)$$

Ahora debemos de proceder a calcular las diferentes transiciones del estado p_0 (2)

$$\delta_N(p_0, n) = \text{clausura} - \epsilon\left(\bigcup_{q \in p_0} \delta_n(q, n)\right) = \text{clausura} - \epsilon(\delta_n(q_0, n) \cup \delta_n(q_1, n) \cup \delta_n(q_{11}, n)) = \quad (3)$$

$$= \text{clausura} - \epsilon(\{\emptyset \cup \{q_2\} \cup \emptyset\}) = \{q_2, q_3, q_4, q_5, q_6, q_{10}, q_{13}\} = p_1 \quad (4)$$

$$\delta_N(p_0, c) = \text{clausura} - \epsilon\left(\bigcup_{q \in p_0} \delta_n(q, c)\right) = \text{clausura} - \epsilon(\delta_n(q_0, c) \cup \delta_n(q_1, c) \cup \delta_n(q_{11}, c)) = \quad (5)$$

$$= \text{clausura} - \epsilon(\{\emptyset \cup \emptyset \cup \{q_{12}\}\}) = \{q_{12}, q_{13}\} = p_2 \quad (6)$$

Ahora procedemos con la transiciones de p_1 (7)

$$\delta_N(p_1, n) = \text{clausura} - \epsilon\left(\bigcup_{q \in p_1} \delta_n(q, n)\right) = \text{clausura} - \epsilon(\delta_n(q_2, n) \cup \delta_n(q_3, n) \cup \delta_n(q_4, n)) \quad (8)$$

$$\cup \delta_n(q_5, n) \cup \delta_n(q_6, n) \cup \delta_n(q_{10}, n) \cup \delta_n(q_{13}, n)) = \text{clausura} - \epsilon\{\emptyset \cup \emptyset \cup \emptyset \cup \emptyset \cup \{q_8\} \cup \emptyset \cup \emptyset\} \quad (9)$$

$$\{q_8, q_9, q_4, q_5, q_6, q_{10}, q_{13}\} = p_3 \quad (10)$$

$$\delta_N(p_1, c) = \text{clausura} - \epsilon\left(\bigcup_{q \in p_1} \delta_n(q, c)\right) = \text{clausura} - \epsilon(\delta_n(q_2, c) \cup \delta_n(q_3, c) \cup \delta_n(q_4, c)) \quad (11)$$

$$\cup \delta_n(q_5, c) \cup \delta_n(q_6, c) \cup \delta_n(q_{10}, c) \cup \delta_n(q_{13}, c)) = \text{clausura} - \epsilon\{\emptyset \cup \emptyset \cup \emptyset \cup \{q_7\} \cup \emptyset \cup \emptyset \cup \emptyset\} \quad (12)$$

$$\{q_7, q_9, q_{10}, q_{13}, q_4, q_5, q_6\} = p_4 \quad (13)$$

Ahora procedemos con las transiciones de p_2 , el cual no posee ninguna (14)

$$\delta_N(p_2, n) = \text{clausura} - \epsilon\left(\bigcup_{q \in p_2} \delta_n(q, n)\right) = \text{clausura} - \epsilon(\delta_n(q_{12}, n) \cup \delta_n(q_{13}, n)) \quad (15)$$

$$= \text{clausura} - \epsilon\{\emptyset \cup \emptyset\} = \emptyset \quad (16)$$

$$\delta_N(p_2, c) = \text{clausura} - \epsilon\left(\bigcup_{q \in p_2} \delta_n(q, c)\right) = \text{clausura} - \epsilon(\delta_n(q_{12}, c) \cup \delta_n(q_{13}, c)) \quad (17)$$

$$= \text{clausura} - \epsilon\{\emptyset \cup \emptyset\} = \emptyset \quad (18)$$

Ahora procedemos con las transiciones de p_3 (19)

$$\delta_N(p_3, n) = \text{clausura} - \epsilon\left(\bigcup_{q \in p_3} \delta_n(q, n)\right) = \text{clausura} - \epsilon(\delta_n(q_8, n) \cup \delta_n(q_9, n) \cup \delta_n(q_4, n)) \quad (20)$$

$$\cup \delta_n(q_5, n) \cup \delta_n(q_6, n) \cup \delta_n(q_{10}, n) \cup \delta_n(q_{13}, n)) = \text{clausura} - \epsilon\{\emptyset \cup \emptyset \cup \emptyset \cup \emptyset \cup \{q_8\} \cup \emptyset \cup \emptyset\} = p_3 \quad (21)$$

$$\delta_N(p_3, c) = \text{clausura} - \epsilon\left(\bigcup_{q \in p_3} \delta_n(q, c)\right) = \text{clausura} - \epsilon(\delta_n(q_8, c) \cup \delta_n(q_9, c) \cup \delta_n(q_4, c)) \quad (22)$$

$$\cup \delta_n(q_5, c) \cup \delta_n(q_6, c) \cup \delta_n(q_{10}, c) \cup \delta_n(q_{13}, c)) = \text{clausura} - \epsilon\{\emptyset \cup \emptyset \cup \emptyset \cup \{q_7\} \cup \emptyset \cup \emptyset \cup \emptyset\} = p_4 \quad (23)$$

Ahora procedemos con las transiciones de p_4 (24)

$$\delta_N(p_4, n) = \text{clausura} - \epsilon\left(\bigcup_{q \in p_4} \delta_n(q, n)\right) = \text{clausura} - \epsilon(\delta_n(q_7, n) \cup \delta_n(q_9, n) \cup \delta_n(q_4, n)) \quad (25)$$

$$\cup \delta_n(q_5, n) \cup \delta_n(q_6, n) \cup \delta_n(q_{10}, n) \cup \delta_n(q_{13}, n)) = \text{clausura} - \epsilon\{\emptyset \cup \emptyset \cup \emptyset \cup \emptyset \cup \{q_8\} \cup \emptyset \cup \emptyset\} = p_3 \quad (26)$$

$$\delta_N(p_4, c) = \text{clausura} - \epsilon\left(\bigcup_{q \in p_4} \delta_n(q, c)\right) = \text{clausura} - \epsilon(\delta_n(q_7, c) \cup \delta_n(q_9, c) \cup \delta_n(q_4, c)) \quad (27)$$

$$\cup \delta_n(q_5, c) \cup \delta_n(q_6, c) \cup \delta_n(q_{10}, c) \cup \delta_n(q_{13}, c)) = \text{clausura} - \epsilon\{\emptyset \cup \emptyset \cup \emptyset \cup \{q_7\} \cup \emptyset \cup \emptyset \cup \emptyset\} = p_4 \quad (28)$$

Siendo el autómata obtenido de la construcción de subconjuntos el que podemos apreciar en la **Figura 3**, en este caso tenemos 5 estados, de los cuales 3 de ellos serían finales. Ahora deberemos de proceder a minimizar el autómata finito determinista.

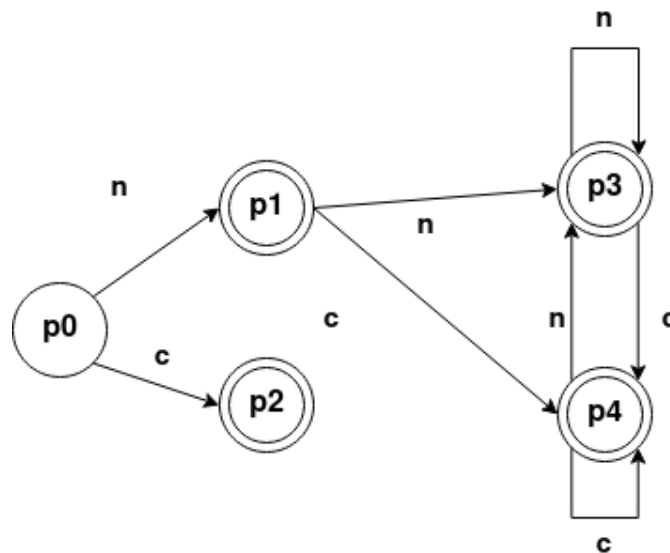


Figura 3: Autómata Finito Determinista - Obtenido por el Algoritmo de Construcción de Subconjuntos

δ	c	n
p_0	p_2	p_1
p_1	p_4	p_3
p_2	-	-
p_3	p_4	p_3
p_4	p_4	p_3

Cuadro 1: Representación de las transiciones del AFD

Minimización de estados:

Para ello en primer lugar agruparemos todos los estados finales en un solo estado y los no finales en otro estado.

- **Estados No Finales:** $a_0 \rightarrow \{p_0\}$
- **Estados Finales:** $a_1 \rightarrow \{p_1, p_2, p_3, p_4\}$

En primer lugar agregaremos todos los estados a $\text{NUEVO} = \{a_0, a_1\}$, marcamos a_0 , y comprobamos si estos estados son "homogeneos". Al contar el estado a_0 de solo un estado, no se puede dividir más. Continuando con a_1 . El cual como podemos apreciar en la **Tabla 2**, podemos apreciar que el estado p_2 no es homogéneo con el resto de estados que componen a_1 siendo necesaria su división en dos estados. De forma que los estados actuales serían:

- $a_0 \rightarrow \{p_0\}$
- $a_1 \rightarrow \{p_1, p_3, p_4\}$
- $a_2 \rightarrow \{p_2\}$

δ	c	n
p_0	a_1	a_1
p_1	a_1	a_1
p_2	-	-
p_3	a_1	a_1
p_4	a_1	a_1

Cuadro 2: Transiciones Paso 1 de Minimización

Ahora reiniciamos las comprobaciones, en este caso al tratar con el estado a_0 , al componerse de un solo estado no se puede dividir más, por lo cual continuamos comprobando el estado a_1 . En este caso como podemos ver en la **Tabla 3**, todos los estados que componen a_1 son homogéneos y el estado a_2 , al componerse de un solo estado este no se puede dividir. De esta forma finalizamos el algoritmo de minimización del AFD. Siendo su representación gráfica la que podemos ver en la **Figura 4**

δ	c	n
p_0	a_2	a_1
p_1	a_1	a_1
p_2	-	-
p_3	a_1	a_1
p_4	a_1	a_1

Cuadro 3: Transiciones Paso 2 de Minimización

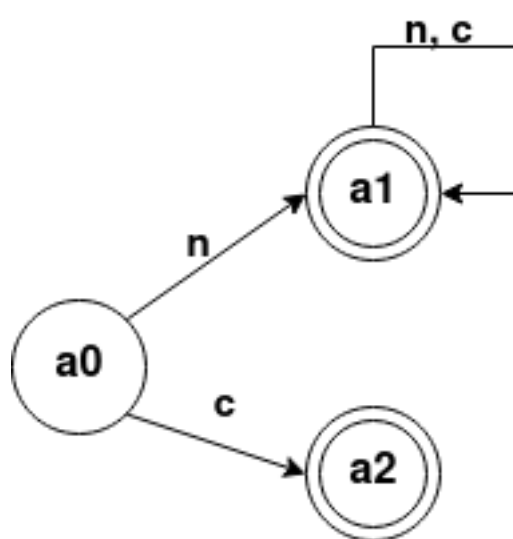


Figura 4: AFD Definitivo tras aplicar la minimización

Análisis descendente predictivo

Considérese la siguiente gramática de contexto libre:

$P = \{$
 1) $S \rightarrow S D$
 2) $S \rightarrow D$
 3) $D \rightarrow T \text{ id } (L) ;$
 4) $T \rightarrow \text{entero}$
 5) $T \rightarrow \text{real}$
 6) $L \rightarrow \epsilon$
 7) $L \rightarrow A$
 8) $A \rightarrow A, \text{id}$
 9) $A \rightarrow \text{id}$
 $\}$

- Esta gramática permite generar declaraciones de prototipos de funciones como, por ejemplo: **real media (entero a, entero b);**
- donde “media”, “a” y “b” son identificadores.

1. Elimina la recursividad por la izquierda y factoriza la gramática por la izquierda.

2. A partir de la gramática obtenida en el apartado “a”:

- Construye los conjuntos “primero” y “siguiente” de los símbolos no terminales.
- Construye la tabla de análisis descendente predictivo.
- Utiliza el método de recuperación de errores de “nivel de frase” para completar la tabla predictiva.
- Utiliza la tabla predictiva para realizar un análisis descendente no recursivo de la siguiente declaración errónea: **real (entero entero a, entero b;**

Resolución:

En primer lugar quitaremos la recursividad, como podemos apreciar tenemos recursividad en las reglas de los estados **S** y **A**. Si procedemos con ello.

1. $S \rightarrow D S'$
2. $S' \rightarrow D S'$
3. $S' \rightarrow \epsilon$
4. $D \rightarrow T \text{ id } (L) ;$
5. $T \rightarrow \text{entero}$
6. $T \rightarrow \text{real}$
7. $L \rightarrow \epsilon$
8. $L \rightarrow A$
9. $A \rightarrow \text{id } A'$
10. $A' \rightarrow , \text{id } A'$
11. $A' \rightarrow \epsilon$

Una vez factorizada, podemos proceder a obtener el conjunto “primero” y “siguiente” de la gramática.

- Regla 1) $S \rightarrow D S'$
- Primero (D) - $\{ \epsilon \} \subseteq \text{Primero } (S)$
- Regla 2) $S' \rightarrow D S'$
- Primero (D) - $\{ \epsilon \} \subseteq \text{Primero } (S')$
- Regla 3) $S' \rightarrow \epsilon$
- $\epsilon \in \text{Primero } (S')$
- Regla 4) $D \rightarrow T \text{ id } (L) ;$
- Primero (D) - $\{ \epsilon \} \subseteq \text{Primero } (D)$

- Regla 5) $T \rightarrow \text{entero}$
- $\text{entero} \in \text{Primero}(T)$
- Regla 6) $T \rightarrow \text{real}$
- $\text{real} \in \text{Primero}(T)$
- Regla 7) $L \rightarrow \epsilon$
- $\epsilon \in \text{Primero}(L)$
- Regla 8) $L \rightarrow A$
- $\text{Primero}(A) - \{\epsilon\} \in \text{Primero}(L)$
- Regla 9) $A \rightarrow \text{id } A'$
- $\text{id} \in \text{Primero}(A)$
- Regla 10) $A' \rightarrow , \text{id } A'$
- $“, ” \in \text{Primero}(A)$
- Regla 11) $A' \rightarrow \epsilon$
- $\epsilon \in \text{Primero}(A)$

Una vez expuesto esto podemos definir todo el conjunto “primero”, el cual podemos verlo en la **Tabla 4**.

Estado	Primero
S	entero, real
S'	entero, real, ϵ
D	entero, real
T	entero, real
L	id
A	id
A'	$\epsilon, “, ”$

Cuadro 4: Conjunto Primero de la gramática obtenida

Ahora procederemos con el conjunto siguiente:

- Regla 1) $S \rightarrow D S'$
 - $\$ \in \text{Siguiente}(S)$
 - $\text{Siguiente}(S) \subseteq \text{Siguiente}(S')$
 - $\text{Primero}(S') - \{\epsilon\} \subseteq \text{Siguiente}(D)$
 - Como $\epsilon \in \text{Primero}(S')$, entonces: $\text{Siguiente}(S) \subseteq \text{Siguiente}(D)$
- Regla 2) $S' \rightarrow D S'$
 - $\text{Siguiente}(S') \subseteq \text{Siguiente}(S')$ (Superflua)
 - $\text{Primero}(S') - \{\epsilon\} \subseteq \text{Siguiente}(D)$ (Repetida)
 - Como $\epsilon \in \text{Primero}(S')$, entonces: $\text{Siguiente}(S') \subseteq \text{Siguiente}(D)$
- Regla 4) $D \rightarrow T \text{id } (L) ;$
 - $“() ” \in \text{Siguiente}(L)$
 - $\text{id} \in \text{Siguiente}(T)$
- Regla 8) $L \rightarrow A$
 - $\text{Siguiente}(L) \subseteq \text{Siguiente}(A)$
- Regla 9) $A \rightarrow \text{id } A'$
 - $\text{Siguiente}(A) \subseteq \text{Siguiente}(A')$
- Regla 10) $A' \rightarrow , \text{id } A'$
 - $\text{Siguiente}(A') \subseteq \text{Siguiente}(A')$ (Superflua)

Siendo el conjunto Siguiente, el que podemos ver en la **Tabla 5**.

Estado	Primero	Siguiente
S	entero, real	\$
S'	entero, real, ϵ	\$
D	entero, real	\$, entero, real
T	entero, real	id
L	id	"")
A	id	"")
A'	ϵ , " , "	"")

Cuadro 5: Conjunto Primero y Siguiente de la gramática obtenida

Una vez contruidos los conjuntos podemos proceder a crear la tabla **predictiva** para el **análisis descendente**. Para ello deberemos de definir las diferentes transiciones por cada una de las reglas.

- Regla 1) $S \rightarrow D S'$
 - Como $\text{Primero}(\alpha) - \{\epsilon\} = \{\text{entero, real}\}$
 - $M[S, \text{entero}] = 1$
 - $M[S, \text{real}] = 1$
- Regla 2) $S' \rightarrow D S'$
 - Como $\text{Primero}(\alpha) - \{\epsilon\} = \{\text{entero, real}\}$
 - $M[S', \text{entero}] = 2$
 - $M[S', \text{real}] = 2$
- Regla 3) $S' \rightarrow \epsilon$
 - Como $\text{Primero}(\alpha) - \{\epsilon\} = \emptyset$, entonces debemos de utilizar el conjunto $\text{Siguiente}(S') = \{\$ \}$
 - $M[S', \$] = 3$
- Regla 4) $D \rightarrow T \text{ id } (L) ;$
 - Como $\text{Primero}(\alpha) - \{\epsilon\} = \{\text{entero, real}\}$
 - $M[D, \text{entero}] = 4$
 - $M[D, \text{real}] = 4$
- Regla 5) $T \rightarrow \text{entero}$
 - Como $\text{Primero}(\alpha) - \{\epsilon\} = \{\text{entero}\}$
 - $M[T, \text{entero}] = 5$
- Regla 6) $T \rightarrow \text{real}$
 - Como $\text{Primero}(\alpha) - \{\epsilon\} = \{\text{real}\}$
 - $M[T, \text{real}] = 6$
- Regla 7) $L \rightarrow \epsilon$
 - Como $\text{Primero}(\alpha) - \{\epsilon\} = \emptyset$, entonces debemos de utilizar el conjunto $\text{Siguiente}(L) = \{ ") " \}$
 - $M[L, ") "] = 7$
- Regla 8) $L \rightarrow A$
 - Como $\text{Primero}(\alpha) - \{\epsilon\} = \{\text{id}\}$
 - $M[L, \text{id}] = 8$
- Regla 9) $A \rightarrow \text{id } A'$
 - Como $\text{Primero}(\alpha) - \{\epsilon\} = \{\text{id}\}$

- $M[A, id] = 9$
- Regla 10) $A' \rightarrow , id A'$
 - Como $\text{Primer}(\alpha) - \{\epsilon\} = \{ ", " \}$
 - $M[A', ", "] = 10$
- Regla 11) $A' \rightarrow \epsilon$
 - Como $\text{Primer}(\alpha) - \{\epsilon\} = \emptyset$, entonces debemos de utilizar el conjunto $\text{Sigui}(\alpha) = \{ ", " \}$
 - $M[A', ", "] = 11$

Una vez definidas todas las transiciones, rellenamos la tabla de predicción.

ESTADO	id	"("	")"	","	","	entero	real	\$
S						1	1	
S'						2	2	3
D						4	4	
T						5	6	
L	8		7					
A	9							
A'			11		10			

Cuadro 6: Tabla predictiva sin métodos de recuperación de errores

Si agregamos los siguientes métodos de recuperación de errores:

- **E1** Falta tipo, Agregar entero en la entrada.
- **E2** Falta id, Agregar id en la entrada.
- **E3** Falta ", ", Agregar ", " en la entrada.
- **E4** Falta ")", Agregar ")" en la entrada.
- **E5** Falta "(", Agregar "(" en la entrada.
- **E6** Símbolo Inesperado, borrar símbolo entrada.
- **E7** Final Inesperado, borrar símbolo de la pila.
- **E8** Falta ", ", Agregar ", " en la entrada.

ESTADO	id	"("	")"	","	","	entero	real	\$
S	E1	E6	E6	E6	E6	1	1	E7
S'	E1	E6	E6	E6	E6	2	2	3
D	E1	E6	E6	E6	E6	4	4	E7
T	E1	E6	E6	E6	E6	5	6	E7
L	8	E6	7	E4	E2	E6	E6	E7
A	9	E6	E2	E6	E2	E6	E6	E7
A'	E3	E6	11	E4	10	E6	E6	E7

Cuadro 7: Tabla predictiva sin métodos de recuperación de errores

VT	id	"("	")"	","	","	entero	real	\$
id	Emparejar	E1	E6	E6	E2	E6	E6	E7
"(" "	E5	Emparejar	E5	E6	E6	E6	E6	E7
")"	E6	E6	Emparejar	E4	E6	E6	E6	E7
"," "	E6	E6	E6	Emparejar	E6	E6	E6	E8
"," "					Emparejar			
entero						Emparejar		
real							Emparejar	
\$	E6	E6	E6	E6	E6	E6	E6	Aceptar

Cuadro 8: Aplicación Tabla Predictiva

Ahora procederemos a reconocer la sentencia **real (entero entero a, entero b;**

Pila	Entrada	Acción
\$ S	real (entero entero id , entero id ; \$	$S \rightarrow D S'$
\$ S' D	real (entero entero id , entero id ; \$	$D \rightarrow T id (L) ;$
\$ S' ;) L (id T	real (entero entero id , entero id ; \$	$T \rightarrow real$
\$ S' ;) L (id real	real (entero entero id , entero id ; \$	Emparejar
\$ S' ;) L (id	(entero entero id , entero id ; \$	E1 Agregar id en la entrada
\$ S' ;) L (id	id (entero entero id , entero id ; \$	Emparejar
\$ S' ;) L (	(entero entero id , entero id ; \$	Emparejar
\$ S' ;) L	entero entero id , entero id ; \$	E6 Eliminar entero de la entrada
\$ S' ;) L	entero id , entero id ; \$	E6 Eliminar entero de la entrada
\$ S' ;) L	id , entero id ; \$	Regla 8) $L \rightarrow A$
\$ S' ;) A	id , entero id ; \$	Regla 9) $A \rightarrow id A'$
\$ S' ;) A' id	id , entero id ; \$	Emparejar
\$ S' ;) A'	, entero id ; \$	Regla 10) $A' \rightarrow , id A'$
\$ S' ;) A' id ,	, entero id ; \$	Emparejar
\$ S' ;) A' id	entero id ; \$	E6 Eliminar entero de la entrada
\$ S' ;) A' id	id ; \$	Emparejar
\$ S' ;) A'	;\$	E4, Falta ")" en la entrada, agregarlo
\$ S' ;) A'	); \$	Regla 11) $A' \rightarrow \epsilon$
\$ S' ;)	); \$	Emparejar
\$ S' ;	;\$	Emparejar
\$ S'	\$	$S' \rightarrow S'$
\$	\$	Aceptar

Cuadro 9: ADP - Análisis sentencia

Análisis sintáctico ascendente SLR

- Considérese la siguiente gramática de contexto libre:

P= {
 1) $S \rightarrow S D$
 2) $S \rightarrow D$
 3) $D \rightarrow \text{Var } L : T ;$
 4) $L \rightarrow L, \text{id}$
 5) $L \rightarrow \text{id}$
 6) $T \rightarrow \text{Integer}$
 7) $T \rightarrow \text{Real}$
 }

- Esta gramática puede generar declaraciones de variables en el lenguaje de programación Pascal, como, por ejemplo: **Var a,b: Integer; Var x,y,z : Real;** donde a,b,x,y,z son identificadores.

1. Construye la colección canónica de LR(0)-elementos del análisis SLR.
2. Dibuja el autómata que reconoce los prefijos viables.
3. Construye los conjuntos primero y siguiente.
4. Construye la tabla de análisis sintáctico SLR: partes acción e ir_a
5. Utiliza el método recuperación de errores de “nivel de frase” para completar la tabla de análisis SLR.
6. Utiliza la tabla SLR para analizar la siguiente declaración errónea: a,b : Integer Integer;

Resolución:

En primer lugar construiremos la colección canónica. Deberemos de ampliar la gramática.

P= {
 1) $S' \rightarrow S$
 2) $S \rightarrow S D$
 3) $S \rightarrow D$
 4) $D \rightarrow \text{Var } L : T ;$
 5) $L \rightarrow L, \text{id}$
 6) $L \rightarrow \text{id}$
 7) $T \rightarrow \text{Integer}$
 8) $T \rightarrow \text{Real}$
 }

$$I_0 = \text{clausura}(\{S' \rightarrow \cdot S\}) = \{S' \rightarrow \cdot S, S \rightarrow \cdot SD, S \rightarrow \cdot D, D \rightarrow \cdot \text{Var } L : T ;\} \quad (29)$$

Ahora debemos de obtener las transiciones de I_0 (30)

$$I_{ra}(I_0, S) = \text{clausura}(\{S' \rightarrow S \cdot, S \rightarrow S \cdot D\}) = \{S' \rightarrow S \cdot, S \rightarrow S \cdot D, D \rightarrow \cdot \text{Var } L : T ;\} = I_1 \quad (31)$$

$$I_{ra}(I_0, D) = \text{clausura}(\{S \rightarrow D \cdot\}) = \{S \rightarrow D \cdot\} = I_2 \quad (32)$$

$$I_{ra}(I_0, \text{Var}) = \text{clausura}(\{D \rightarrow \text{Var} \cdot L : T ;\}) = \{D \rightarrow \text{Var} \cdot L : T ;, L \rightarrow \cdot L, \text{id}, L \rightarrow \cdot \text{id}\} = I_3 \quad (33)$$

Ahora deberemos de realizar las transiciones de I_1 (34)

$$I_{ra}(I_1, \text{Var}) = \text{clausura}(\{D \rightarrow \text{Var} \cdot L : T ;\}) = I_3 \quad (35)$$

$$I_{ra}(I_1, D) = \text{clausura}(\{S \rightarrow SD \cdot\}) = I_4 \quad (36)$$

Ahora deberemos de realizar las transiciones de I_2 , el cual no posee ninguna (37)

Ahora deberemos de realizar las transiciones de I_3 (38)

$$Ir_a(I_3, L) = \text{clausura}(\{D \rightarrow \mathbf{Var} L : T; , L \rightarrow L \cdot id\}) = \{D \rightarrow \mathbf{Var} L : T; , L \rightarrow L \cdot id\} = I_5 \quad (39)$$

$$Ir_a(I_3, id) = \text{clausura}(\{L \rightarrow id \cdot\}) = \{L \rightarrow id \cdot\} = I_6 \quad (40)$$

Ahora deberemos de realizar las transiciones de I_4 , el cual no posee ninguna (41)

Ahora deberemos de realizar las transiciones de I_5 (42)

$$Ir_a(I_5, :) = \text{clausura}(\{D \rightarrow \mathbf{Var} L : T; \}) = \{D \rightarrow \mathbf{Var} L : T; , T \rightarrow \cdot Integer, T \rightarrow \cdot Real\} = I_7 \quad (43)$$

$$Ir_a(I_5, ",") = \text{clausura}(\{L \rightarrow L \cdot id\}) = \{L \rightarrow L \cdot id\} = I_8 \quad (44)$$

Ahora deberemos de realizar las transiciones de I_6 No posee transiciones (45)

Ahora deberemos de realizar las transiciones de I_7 (46)

$$Ir_a(I_7, T) = \text{clausura}(\{D \rightarrow \mathbf{Var} L : T; \}) = \{D \rightarrow \mathbf{Var} L : T; \} = I_9 \quad (47)$$

$$Ir_a(I_7, Integer) = \text{clausura}(\{T \rightarrow Integer \cdot\}) = \{T \rightarrow Integer \cdot\} = I_{10} \quad (48)$$

$$Ir_a(I_7, Real) = \text{clausura}(\{T \rightarrow Real \cdot\}) = \{T \rightarrow Real \cdot\} = I_{11} \quad (49)$$

Ahora deberemos de realizar las transiciones de I_8 (50)

$$Ir_a(I_8, id) = \text{clausura}(\{L \rightarrow L \cdot id\}) = \{L \rightarrow L \cdot id\} = I_{12} \quad (51)$$

Ahora deberemos de realizar las transiciones de I_9 (52)

$$Ir_a(I_9, :) = \text{clausura}(\{D \rightarrow \mathbf{Var} L : T; \cdot\}) = \{D \rightarrow \mathbf{Var} L : T; \cdot\} = I_{13} \quad (53)$$

Ahora deberemos de realizar las transiciones de I_{10} No posee transiciones (54)

Ahora deberemos de realizar las transiciones de I_{11} No posee transiciones (55)

Ahora deberemos de realizar las transiciones de I_{12} No posee transiciones (56)

Ahora deberemos de realizar las transiciones de I_{13} No posee transiciones (57)

En la **Figura 5**, podemos apreciar el autómata que reconoce los prefijos viables. Ahora deberemos de construir los conjuntos “**primero**” y “**siguiente**”.

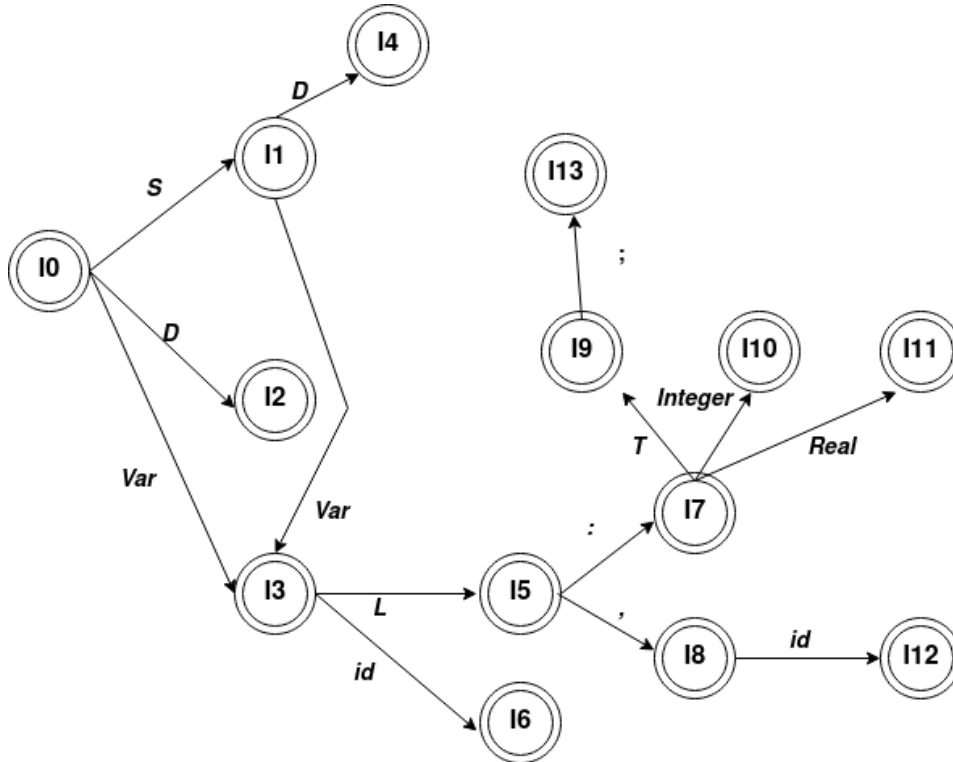


Figura 5: Autómata que reconoce los prefijos viables

- Regla 1') $S' \rightarrow S$
- Primero (S) $\subseteq$ Primero(S')
- Regla 1) $S \rightarrow S D$
- Primero (S) $\subseteq$ Primero(S)
- Regla 2) $S \rightarrow D$
- Primero (D) $\subseteq$ Primero(S)
- Regla 3) $D \rightarrow \text{Var } L : T ;$
- $\text{Var} \in \text{Primero}(D)$
- Regla 4) $L \rightarrow L, \text{id}$
- Primero(L) $\subseteq$ Primero(L)
- Regla 5) $L \rightarrow \text{id}$
- $\text{id} \in \text{Primero}(L)$
- Regla 6) $T \rightarrow \text{Integer}$
- $\text{Integer} \in \text{Primero}(T)$
- Regla 7) $T \rightarrow \text{Real}$
- $\text{Real} \in \text{Primero}(T)$

Estado	Primero
S'	Var
S	Var
D	Var
L	id
T	real, integer

Cuadro 10: Conjunto Primero

Ahora procederemos con el conjunto siguiente:

- Regla 1') $S' \rightarrow S$
 - $\$ \in \text{Siguiente}(S')$
 - $\text{Siguiente}(S') \subseteq \text{Siguiente}(S)$
- Regla 1) $S \rightarrow S D$
 - $\text{Siguiente}(D) \subseteq \text{Siguiente}(S)$
- Regla 2) $S \rightarrow D$
 - $\text{Siguiente}(D) \subseteq \text{Siguiente}(S)$ (Repetida)
- Regla 3) $D \rightarrow \text{Var } L : T ;$
 - $; \in \text{Siguiente}(T)$
 - $: \in \text{Siguiente}(L)$
- Regla 4) $L \rightarrow L, \text{id}$
 - $, \in \text{Siguiente}(L)$

Estado	Primero	Siguiente
S'	Var	\$
S	Var	\$
D	Var	\$
L	id	, :
T	real, integer	;

Cuadro 11: Conjunto Primero y Siguiente de la gramática obtenida

Ahora deberemos de construir la tabla de análisis sintáctico SLR

ESTADO	Var	:	;	,	id	real	integer	\$	S	D	L	T
I_0	d3								1	2		
I_1	d3							Aceptar		4		
I_2								r2				
I_3					d6						5	
I_4								r1				
I_5		d7		d8								
I_6		r5		r5								
I_7						d11	d10					9
I_8					d12							
I_9			d13									
I_{10}			r6									
I_{11}			r7									
I_{12}		r4		r4								
I_{13}								r3				

Cuadro 12: Tabla de análisis sintáctico SLR

Una vez construida la tabla “base” deberemos de agregar las diferentes funciones de recuperación de errores de **nivel de frase**.

- **E1** Falta var, insertar var en la entrada.
- **E2** Falta id, insertar id en la entrada.
- **E3** Falta “,” , insertar “,” en la entrada.
- **E4** Falta “.” , insertar “.” en la entrada.
- **E5** Falta tipo , insertar integer en la entrada.
- **E6** Falta “;” , insertar “;” en la entrada.
- **E7** Símbolos inesperado, borrar símbolo de la entrada.
- **E8** Final inesperado, borrar símbolo de la pila.

ESTADO	Var	:	;	,	id	real	integer	\$	S	D	L	T
I_0	d3	E7	E7	E7	E1	E7	E7	E8	1	2		
I_1	d3	E7	E7	E7	E1	E7	E7	Aceptar		4		
I_2	E7	E7	E7	E7	E7	E7	E7	r2				
I_3	E7	E2	E7	E7	d6	E7	E7	E7			5	
I_4	E7	E7	E7	E7	E7	E7	E7	r1				
I_5	E7	d7	E7	d8	E3	E4	E4	E7				
I_6	E7	r5	E7	r5	E7	E7	E7	E7				
I_7	E7	E7	E5	E7	E7	d11	d10	E7				9
I_8	E7	E1	E7	E1	d12	E7	E7	E7				
I_9	E6	E7	d13	E7	E7	E7	E7	E6				
I_{10}	E7	E7	r6	E7	E7	E7	E7	E6				
I_{11}	E7	E7	r7	E7	E7	E7	E7	E6				
I_{12}	E7	r4	E7	r4	E7	E7	E7	E7				
I_{13}	E7	E7	E7	E7	E7	E7	E7	r3				

Cuadro 13: Tabla de análisis sintáctico SLR

Una vez finalizada la tabla, podemos proceder a analizar la sentenecia: **a , b : Integer Integer ;**

Pila	Entrada	Acción
0	id , id : Integer Integer ; \$	E1 ->Falta var
0	var id , id : Integer Integer ; \$	d3
0 var 3	id , id : Integer Integer ; \$	d6
0 var 3 id 6	, id : Integer Integer ; \$	r5 L → id
0 var 3 L 5	, id : Integer Integer ; \$	d8
0 var 3 L 5 , 8	id : Integer Integer ; \$	d12
0 var 3 L 5 , 8 id 12	: Integer Integer ; \$	r4 L → L, id
0 var 3 L 5	: Integer Integer ; \$	d7
0 var 3 L 5 : 7	Integer Integer ; \$	d10
0 var 3 L 5 : 7 Integer 10	Integer ; \$	E7 ->Símbolos inesperado, borramos de la entrada
0 var 3 L 5 : 7 Integer 10	; \$	r6 T → integer
0 var 3 L 5 : 7 T 9	; \$	d13
0 var 3 L 5 : 7 T 9 ; 13	\$	r3 D → Var L : T ;
0 D 2	\$	r2 S → D
0 S 1	\$	Aceptar

Cuadro 14: SLR - Análisis sentencia

Diseño de una gramática de contexto libre

- Diseña una gramática que permita generar declaraciones de tipos registros usando la sintaxis del lenguaje de programación Pascal.

Ejemplo de Gramática

```
Type TipoPersona = Record  
    Nombre, Apellidos : string;  
    Direccion : array [1 .. 3] of string;  
    Telefono : Integer;  
End;
```

Un ejemplo de gramática podría ser:

1. $S \rightarrow D S$
2. $S \rightarrow \epsilon$
3. $D \rightarrow \text{type id = record L end};$
4. $L \rightarrow I : T ; L'$
5. $L' \rightarrow I : T ; L'$
6. $L' \rightarrow \epsilon$
7. $I \rightarrow \text{id } I'$
8. $I' \rightarrow , \text{id } I'$
9. $I' \rightarrow \epsilon$
10. $T \rightarrow \text{string}$
11. $T \rightarrow \text{Integer}$
12. $T \rightarrow \text{array [numero .. numero] of T}$